



2.15. Основы алгоритмов, часть 2

Всем привет!

На связи шпаргалка урока 2.15. Основы алгоритмов, часть 2. В сегодняшней шпаргалке рассмотрим некоторые базовые алгоритмы, а именно простые сортировки и поиск в массивах; напишем код этих алгоритмов и разберем их пошаговую работу.

Время прочтения: 13 минут.

Сортировка

Пузырьковая сортировка

Сортировка выбором

Сортировка вставкой

Поиск

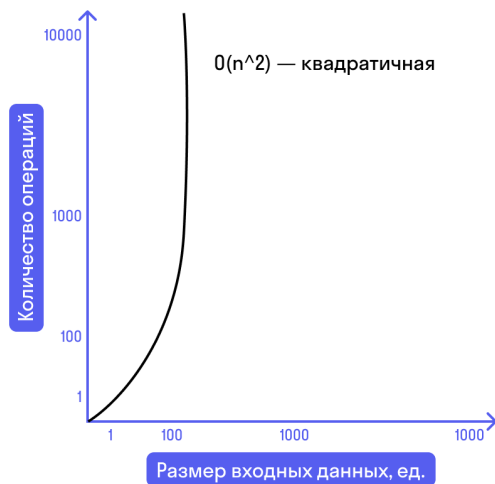
Линейный поиск

Бинарный поиск

Сортировка

В качестве сортировок мы сегодня рассмотрим три вариации, которые имеют квадратичную сложность. Но при общей сложности они имеют разное время выполнения.

Квадратичная сложность



Две из трех рассматриваемых сортировок требуют небольшого утилити-метода по перемещению элементов местами.

▼ Напишем этот метод для избавления от дублирующего кода.

```
private static void swapElements(int[] arr, int indexA, int indexB) {  
    int tmp = arr[indexA];  
    arr[indexA] = arr[indexB];  
    arr[indexB] = tmp;  
}
```

Никакой сложной логики в коде выше нет, но для улучшения читабельности и соблюдения принципа DRY (Don't repeat yourself) мы не будем дублировать его, а вынесем в отдельный метод.

С огромной долей вероятности вам не придется в рабочих задачах реализовывать эти сортировки, однако на собеседованиях вас могут попросить реализовать или хотя бы объяснить любую из них.

Также те алгоритмы, что представлены ниже, позволят вам в очередной раз убедиться в том, как сильно могут различаться решения для одних и тех же задач в программировании.

Приступим к самим сортировкам.

Пузырьковая сортировка

Начнем с упомянутой в прошлой шпаргалке пузырьковой сортировки.

Данный вид сортировки является самым простым и популярным среди начинающих разработчиков.

▼ Код

```
public static void sortBubble(int[] arr) {  
    for (int i = 0; i < arr.length - 1; i++) {  
        for (int j = 0; j < arr.length - 1 - i; j++) {  
            if (arr[j] > arr[j + 1]) {  
                swapElements(arr, j, j + 1);  
            }  
        }  
    }  
}
```

Рассмотрим пошагово работу пузырьковой сортировки:

1. Объявляется первый цикл, который управляет количеством проходов по массиву.

Количество шагов цикла равно количеству элементов массива минус 1.

2. Объявляется внутренний цикл, который уже проходит по всем элементам и осуществляет сравнение текущего элемента со следующим.

Количество шагов внутреннего цикла равняется

$\text{size} - 1 - i$

то есть *количество элементов массива минус один и минус i* , где i — переменная из первого (внешнего) цикла.

Шаги начинаются с 0, т. к. по этой переменной будут обращения к массиву и его элементам.

3. На каждом шаге внутренний цикл сравнивает текущий элемент со следующим и, если текущий больше следующего, элементы меняются местами.

4. В результате каждого из обходов на последнюю позицию из внутреннего цикла устанавливается максимальный элемент.

Взглянем на шаги работы алгоритма на примере массива чисел {3, 2, 1, 4}.

▼ Шаги

```
// 1-я итерация: i = 0; j от 0 до 2; arr = {3, 2, 1, 4}
// 1-й шаг: j = 0; arr[j] = 3, arr[j + 1] = 2; swap; arr = {2, 3, 1, 4}
// 2-й шаг: j = 1; arr[j] = 3, arr[j + 1] = 1; swap; arr = {2, 1, 3, 4}
// 3-й шаг: j = 2; arr[j] = 3, arr[j + 1] = 4; arr = {2, 1, 3, 4}
// 2-я итерация: i = 1; j от 0 до 1; arr = {2, 1, 3, 4}
// 1-й шаг: j = 0; arr[j] = 2, arr[j + 1] = 1; swap; arr = {1, 2, 3, 4}
// 2-й шаг: j = 1; arr[j] = 2, arr[j + 1] = 3; arr = {1, 2, 3, 4}
// 3-я итерация: i = 2; j = 0; arr = {1, 2, 3, 4}
// 1-й шаг: j = 0; arr[j] = 1; arr[j + 1] = 2; arr = {1, 2, 3, 4}
// 4-я итерация: i = 3; внутренний цикл не запускается, т. е. j = 0,
// а допустимый интервал значений j < 0;
```

Как вы могли видеть выше, массив был отсортирован уже на 1-м шаге 2-й итерации, но цикл не прервался.

Циклы в любом случае пройдут оставшиеся шаги, но перемещений уже не будет, т. к. массив отсортирован.

Сортировка выбором

Данный алгоритм является еще одной сортировкой с квадратичной сложностью.

Его основная идея заключается в том, что на каждой итерации находится минимальный элемент и перемещается в самую левую ячейку текущей итерации.

Элемент из самой левой ячейки перемещается туда, где был минимальный элемент до перемещения.

Т. е. в отличие от пузырьковой сортировки swap происходит не постоянно, а 1 раз за итерацию.

▼ Код

```

public static void sortSelection(int[] arr) {
    for (int i = 0; i < arr.length - 1; i++) {
        int minElementIndex = i;
        for (int j = i + 1; j < arr.length; j++) {
            if (arr[j] < arr[minElementIndex]) {
                minElementIndex = j;
            }
        }
        swapElements(arr, i, minElementIndex);
    }
}

```

Рассмотрим пошагово работу сортировкой выбором:

1. Объявляется первый цикл, который проходит по всем ячейкам массива, кроме последней.
2. На каждом проходе цикла запоминается индекс элемента из текущей ячейки (i) в качестве минимального в переменную minElementIndex.
3. Запускается внутренний цикл. Он проходит по ячейкам, которые ограничены интервалом от i + 1 до последнего элемента включительно.
4. Внутренний цикл сравнивает все элементы с минимальным и, если текущий элемент меньше, присваивает его индекс переменной minElementIndex.
5. По завершению прохода внутреннего цикла элемент по минимальному индексу обменивается местами с элементом по индексу i.

Взглянем на шаги работы алгоритма на примере массива чисел {2, 3, 1}.

▼ Шаги

```

// 1-я итерация: i = 0; minElementIndex = 0; arr = {2, 3, 1}
// 1-й шаг: j = 1; arr[j] = 3, arr[minElementIndex] = 2; minElementIndex не меняется;
// 2-й шаг: j = 2; arr[j] = 1, arr[minElementIndex] = 2; minElementIndex = 2;
// swap arr[i] и arr[minElementIndex]; arr = {1, 3, 2}
// 2-я итерация: i = 1; minElementIndex = 1; arr = {1, 3, 2}
// 1-й шаг: j = 2; arr[j] = 2; arr[minElementIndex] = 3; minElementIndex = 2;
// swap arr[i] и arr[minElementIndex]; arr = {1, 2, 3}

```

Один своп на всю итерацию позволяет не свопать элементы на каждом шаге, что ускоряет работу.

Данная сортировка показывает большую скорость работы, чем пузырьковая, хотя и имеет ту же квадратичную сложность.

Сортировка вставкой

Следующим видом сортировки с квадратичной сложностью является сортировка вставкой.

Идея данного алгоритма состоит в том, что на каждом шаге цикла вычисляется определенная позиция, а все элементы слева сдвигаются на один вправо до тех пор, пока не будет найдена корректная позиция для найденного элемента. Затем этот элемент устанавливается в найденную позицию.

▼ Код

```
public static void sortInsertion(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int temp = arr[i];
        int j = i;
        while (j > 0 && arr[j - 1] >= temp) {
            arr[j] = arr[j - 1];
            j--;
        }
        arr[j] = temp;
    }
}
```

Рассмотрим пошагово алгоритм выше:

1. Объявляется цикл, который проходит по всем элементам массива, кроме первого.
2. Запоминается элемент, который находится по текущему индексу.
3. Объявляется переменная, которая будет идти в обратном направлении.
4. Запускается цикл, который должен смещаться влево до тех пор, пока не будет найден элемент меньше запомненного в шаге 2 или пока не будет

достигнут 0 (нулевой) элемент массива.

5. Этот цикл смещает все элементы, которые больше запомненного в шаге 2, на 1 ячейку вправо.
6. В освободившуюся позицию устанавливается значение из шага 2.

Взглянем на шаги на примере массива {3, 2, 1}.

▼ Шаги

```
// 1-я итерация: i = 1; temp = 2; arr = {3, 2, 1}
// 1-й шаг: j = 1; arr[j] = 2, arr[j - 1] = 3; arr = {3, 3, 1}
// swap: j = 0; arr[j] = temp; arr = {2, 3, 1}
// 2-я итерация: i = 2; temp = 1; arr = {2, 3, 1}
// 1-й шаг: j = 2; arr[j] = 1; arr[j - 1] = 3; swap; arr = {2, 3, 3}
// 2-й шаг: j = 1; arr[j] = 3; arr[j - 1] = 2; swap; arr = {2, 2, 3}
// swap: j = 0; arr[j] = temp; arr = {1, 2, 3}
```

Данный алгоритм тоже не осуществляет свапов «всех со всеми», что позволяет ему выигрывать в скорости выполнения относительно пузырька.

Поиск

Рассмотрим другую частую задачу, а именно поиск элемента.

Для сравнения приведены линейный поиск, который вы уже реализовывали много раз, и бинарный поиск.

Линейный имеет линейную сложность, бинарный — логарифмическую сложность.

Линейный поиск

С этим видом поиска вы уже знакомы.

Это простой проход через все элементы массива, который сравнивает каждый элемент с необходимым.

▼ Код

```
public static boolean contains(int[] arr, int element) {  
    for (int i : arr) {  
        if (i == element) {  
            return true;  
        }  
    }  
    return false;  
}
```

Рассмотри код пошагово:

1. Запускается цикл, который проходит по всем элементам массива.
2. На каждом шаге цикла сравнивается текущий элемент с тем, который планируется найти.
3. Если объект найден, возвращается true.
4. Если цикл закончил свою работы и метод не был прерван возвратом true, значит, элемент не найден и возвращается false.

Бинарный поиск

Данный вид поиска обсуждался в прошлой шпаргалке, но код его не показывался.

Основной идеей алгоритма является «отсечение» половины всех данных, которые точно не соответствуют разыскиваемому значению.

Именно поэтому бинарный поиск возможен только в случае отсортированных данных.

▼ Код

```
public static boolean contains(int[] arr, int element) {  
    int min = 0;  
    int max = arr.length - 1;  
  
    while (min <= max) {  
        int mid = (min + max) / 2;
```



```

        if (element == arr[mid]) {
            return true;
        }

        if (element < arr[mid]) {
            max = mid - 1;
        } else {
            min = mid + 1;
        }
    }
    return false;
}

```

Рассмотрим по шагам:

1. Первым делом в переменные присваиваются минимальный и максимальный индексы массива.
2. Далее запускается цикл, который работает до тех пор, пока минимальный индекс не становится больше, чем максимальный, т. е. есть, что проверять.
3. В цикле осуществляется вычисление середины проверяемого интервала.
4. Затем центральный элемент интервала сравнивается со значением. Если проверка проходит, возвращается true.
5. Если проверка из предыдущего шага не прошла, проверяется условие, является ли искомый элемент больше или меньше среднего элемента.
6. В зависимости от проверки из шага 5 min или max становятся равны среднему элементу ± 1 , т. к. проверять центральный элемент уже не требуется.

Это позволяет сократить интервал поиска на каждом шаге ровно в 2 раза.

Благодаря такой нехитрой схеме достигается логарифмическая сложность поиска, но дополнительные ограничения в виде сортировки данных может сделать линейный поиск более производительным, т. к. даже самые быстрые сортировки имеют сложность больше, чем сложность линейного поиска.

Однако если поиск будет производиться часто, имеет смысл один раз осуществить сортировку и уже затем много раз искать данные с помощью бинарного поиска.